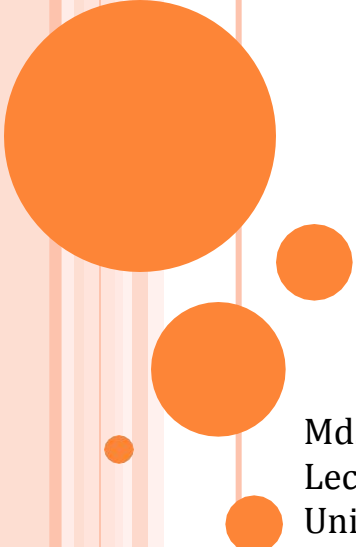


CLASS & OBJECT



Md. Shafqat Talukder Rakin
Lecturer, Department of CSE,
United International University
Email id : shafqat@cse.uiu.ac.bd

Acknowledged: Tanjina Helaly

CONTENT

- Class & Object.
- Initializing Fields/Instance variables.
- Scope & Lifetime of Variable.
- Arrays
- Parameter Passing
- Garbage Collection
- Package
- Access Modifier
- Recursion
- Variable Argument - vararg



TWO PARADIGMS IN OOP

- All computer programs consist of two elements:
 - code and data.
- A program can be conceptually organized around its code or around its data.
- That is, some programs are written around “what is happening” and others are written around “who is being affected.”
- These are the two paradigms that govern how an OOP program is constructed.
- In Procedural language data and operation/code are separate
- Example (using C)
 - Student – 4 fields: name, id, cgpa, creditCompleted
 - Update cgpa – need a function to calculate the new cgpa



CLASS & OBJECT

□ Object

- An object is a software bundle of related state and behavior.
- Software objects are often used to model the real-world objects that you find in everyday life.

□ Class

- A class is a **blueprint** or **prototype** from which objects are created.
- It **defines** what should be in each object and how each object should behave.



WHAT IS CLASS

- the class is the **basis** for object-oriented programming in Java.
- it defines a **new data type**.
- Once defined, this new type can be used to create objects of that type.
 - Thus, a class is a *template for an object*,
 - and an object is an *instance of a class*.
 - Because an object is an instance of a class, you will often see the two words *object* and *instance* used interchangeably



CLASS MEMBERS

- Class contains the following 2 members
 - instance variables(fields/properties/attributes)
 - **each instance of the class** (that is, each object of the class) **contains its own copy of these variables.**
 - Thus, the data for one object is separate and unique from the data for another.
 - Methods(action/behavior)
 - *The code is contained within methods*
 - In most classes, the instance variables are acted upon and accessed by the methods defined for that class.
 - Thus, as a general rule, it is the methods that determine how a class' data can be used.



THE GENERAL FORM OF A CLASS

```
class classname {  
    datatype instance-variable1;  
    datatype instance-variable2;  
    // ...  
    datatype instance-variableN;  
  
    returntype methodname1(parameter-list) {  
        // body of method  
    }  
  
    returntype methodname2(parameter-list) {  
        // body of method  
    }  
  
    // ...  
  
    returntype methodnameN(parameter-list) {  
        // body of method  
    }  
}
```



Class Example

Bank Account Class:

Create a BankAccount class with attributes for account holder's name(string type), account id(string type), and balance(double type). Implement methods for depositing and withdrawing funds. Test your class by creating instances of bank accounts and performing transactions.



A SIMPLE CLASS

```
public class BankAccount {  
    // Instance variables  
    public String name;  
    public String id; public  
    double balance;  
  
    // Methods  
    public void deposit(double amount){  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount){ if  
        (amount<balance)  
        balance -= amount;  
    }  
}
```



CREATE OBJECT AND ACCESS MEMBERS

▣ Creating objects – *use **new** keyword*

- `ClassName refVariable = new ClassName(parameter-list);`

▣ To access instance variables and methods - *use **dot operator**(.)*

● Accessing instance variables

- ▣ `refVariable.instance_variable = value;`
- ▣ Or
- ▣ `datatype value = refVariable.instance_variable;`

● Calling methods

- ▣ `refVariable.methodName(parameter-list);`



CREATE OBJECT AND ACCESS MEMBERS

```
public class TestBankAccount {  
    public static void main(String[] args) {  
        // Creating objects  
        BankAccount account = new BankAccount();  
  
        // Assigning instance variables  
        account.name = "Rashid";  
        account.id = "1000500";  
        account.balance = 1000;  
  
        // Print balance  
        System.out.println("Balance before deposit: " + account.balance);  
  
        // Calling methods  
        account.deposit(2000);  
  
        // Print balance  
        System.out.println("Balance after deposit: " + account.balance);  
    }  
}
```

Output:

Balance before deposit: 1000.0

Balance after deposit: 3000.0



CREATE OBJECT AND ACCESS MEMBERS

```
public class TestBankAccount {
```

```
    public static void main(String[] args) {
```

```
        // Creating objects
```

```
        BankAccount account = new BankAccount();
```

Use "new" keyword
to create object

```
        // Assigning values to instance variables
```

```
        account.name = "Rashid";
```

```
        account.id = "1000500";
```

```
        account.balance = 1000;
```

Use "dot operator" (.) to access
attributes and methods.

```
        // Print balance
```

```
        System.out.println("Balance before deposit: " + account.balance);
```

```
        // Calling methods
```

```
        account.deposit(2000);
```

```
        // Print balance
```

```
        System.out.println("Balance after deposit: " + account.balance);
```

```
    }
```

```
}
```

Output:

Balance before deposit: 1000.0

Balance after deposit: 3000.0



REFERENCE VARIABLE

BankAccount account;

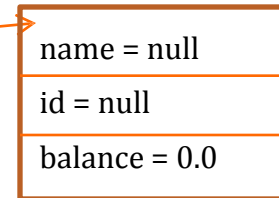
account



account = new BankAccount();

account

reference



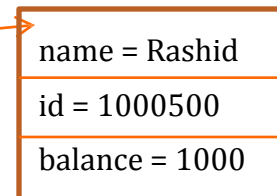
account.name = "Rashid";

account.id = "1000500";

account.balance = 1000;

account

reference



CREATE OBJECT AND ACCESS MEMBERS

```
public class TestBankAccount {  
    public static void main(String[] args) {  
        // Creating objects  
        BankAccount accountR = new BankAccount();  
        BankAccount accountK = new BankAccount();  
  
        // Assigning values to instance variables  
        accountR.name = "Rashid";  
        accountR.balance = 1000;  
  
        accountK.name = "Kashem";  
        accountK.balance = 1000;  
  
        // Calling methods  
        accountK.deposit(2000);  
  
        // Print balance of both account  
        System.out.println("Kashem's balance: " + accountK.balance);  
        System.out.println("Rashid's balance: " + accountR.balance);  
    }  
}
```

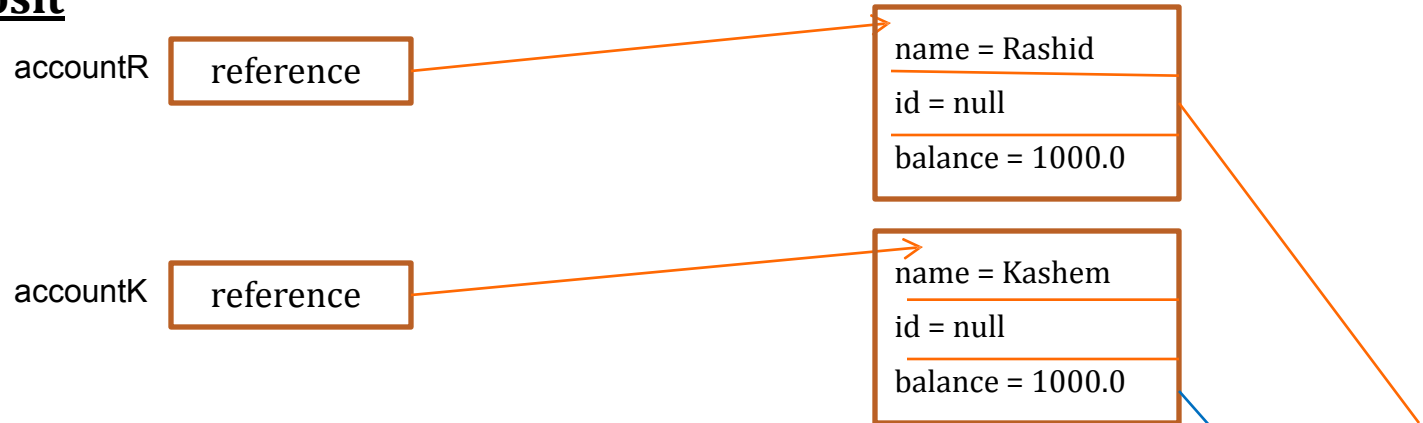
Output:

```
Kashem's balance: 3000.0  
Rashid's balance: 1000.0
```

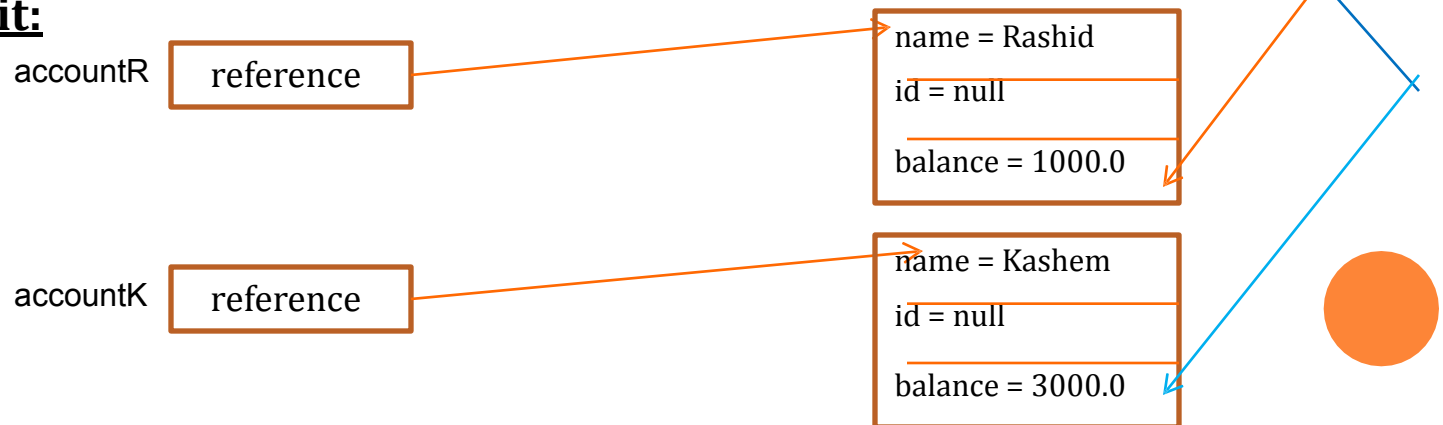


REFERENCE VARIABLE

Before deposit



After deposit:



INITIALIZING FIELDS/INSTANCE VARIABLES



INITIALIZING FIELDS

- There are three ways in Java to give a field an initial value:
 - Direct Assignment
 - Instance Initialization Block
 - Constructors



1.DIRECT ASSIGNMENT

```
public class BankAccount {  
    // Instance variables  
    public String name;  
    public String id;  
    public double balance = 100.0; // direct assignment  
  
    // Methods  
    public void deposit(double amount){  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount){ if  
        (amount<balance)  
        balance -= amount;  
    }  
}
```



2.INSTANCE INITIALIZATION BLOCK

- Instance initialization blocks are indicated by blocks of code inside the class, but outside any method.
- *Whenever an object is created from a class the code in each instance initialization block is executed.*
- If there is more than one instance initialization block they are executed in order, from top to bottom of the class.
- Use initialization blocks *when the initialization cannot be done in a simple assignment and needs no extra input parameters.*
- Direct assignment and constructors are used far more often than initialization blocks.



2.INSTANCE INITIALIZATION BLOCK

```
public class BankAccount {  
    // Instance variables  
    public String name;  
    public String id; public  
    double balance;  
  
    // Methods  
    public void deposit(double amount){  
        balance = balance + amount;  
    }  
    public void withdraw(double amount){ if  
        (amount<balance)  
        balance -= amount;  
    }  
  
    {    // Instance Initialization Block  
        id = new Random().nextInt(99999) + "";  
        balance = 100.0;  
    }  
}
```



3.CONSTRUCTOR

□ A constructor

- *Allocate space for instance variables.*
- *initializes an object(its instance variables) immediately upon creation.*

□ Syntax:

- *It has the same name as the class.*
- syntactically similar to a method.
 - Except has no return type. Not even **void**.
 - **This is because the implicit return type of a class' constructor is the class type itself.**



3.CONSTRUCTOR

- When called:
 - No explicit call
 - It is automatically called when the object is created, before the **new operator** completes.
- What should go inside Constructor
 - Normally the instance variables are initialized inside the constructor.
 - Or
 - any set-up code



3.CONSTRUCTOR - EXAMPLE

```
import java.util.Random;
public class BankAccount {
    // Instance variables
    public String name;
    public String id; public
    double balance;

    // Constructor without parameter
    public BankAccount(){
        id = new Random().nextInt(99999) + "";
        // name and balance will get default value
    }

    // Constructor with parameter
    public BankAccount(String _name, String _id, double _balance){
        name = _name;
        id = _id;
        balance = _balance;
    }

    public static void main(String[] args)
    {
        BankAccount ba = new BankAccount("Rashid", "1000500", 1000.0);
    }
}
```



3.DEFAULT CONSTRUCTOR

- When you do not explicitly define a constructor for a class,
 - then Java creates a default constructor for the class.
 - This is why the first BankAccount class code worked even though **that did not define a constructor.**

- **The default** constructor automatically initializes all instance variables to their default values,
 - Default value
 - Boolean -> false;
 - All primitive except boolean -> 0 (zero)
 - Reference type -> null

- Once you define your own constructor, the default constructor is no longer used.



“THIS” KEYWORD

- refer to the *current object*.
- That is, **this is always a reference to the object on which the method was invoked.**
- **You can use this anywhere a reference to an object of the current class' type is permitted.**



CONSTRUCTOR - EXAMPLE

```
import java.util.Random;
public class BankAccount {
    // Instance variables
    public String name;
    public String id; public
    double balance;

    // Constructor without parameter
    public BankAccount(){
        id = new Random().nextInt(99999) + "";
        // name and balance will get default value
    }

    // Constructor with parameter
    public BankAccount(String _name, String _id, double _balance){
        name = _name;
        id = _id;
        balance = _balance;
    }

    public static void main(String[] args)
    {
        BankAccount ba = new BankAccount("Rashid", "1000500", 1000.0);
    }
}
```



CONSTRUCTOR - EXAMPLE

```
import java.util.Random;
public class BankAccount {
    // Instance variables
    public String name;
    public String id; public
    double balance;

    // Constructor without parameter
    public BankAccount(){
        id = new Random().nextInt(99999) + "";
        // name and balance will get default value
    }

    // Constructor with parameter
    public BankAccount(String name, String id, double balance){
        name = name;
        id = id;
        balance = balance;
    }

    public static void main(String[] args)
    {
        BankAccount ba = new BankAccount("Rashid", "1000500", 1000.0);
    }
}
```



CONSTRUCTOR - EXAMPLE

```
import java.util.Random;
public class BankAccount {
    // Instance variables
    public String name;
    public String id; public
    double balance;

    // Constructor without parameter
    public BankAccount(){
        id = new Random().nextInt(99999) + "";
        // name and balance will get default value
    }

    // Constructor with parameter
    public BankAccount (String name, String id, double balance) {
        name = name;
        id = id;
        balance = balance;
    }

    public static void main(String[] args)
    {
        BankAccount ba = new BankAccount("Rashid", "1000500", 1000.0);
    }
}
```

Variable on left is the same as right
What are we doing here
Which variables we are referring to

CONSTRUCTOR - EXAMPLE

```
import java.util.Random;
public class BankAccount {
    // Instance variables
    public String name;
    public String id; public
    double balance;

    // Constructor without parameter
    public BankAccount(){
        id = new Random().nextInt(99999) + "";
        // name and balance will get default value
    }

    // Constructor with parameter
    public BankAccount(String name, String id, double balance){
        this.name = name;
        this.id = id;
        this.balance = balance;
    }

    public static void main(String[] args)
    {
        BankAccount ba = new BankAccount("Rashid", "1000500", 1000.0);
    }
}
```



Spring 2024 Question 1

Consider the following codes:

```
class Point2D
{
    int x, y;
    public Point2D(int x, int y){
        this.x = x;
        this.y = y;
        System.out.println("Point2D constructor");
    }
    public String Display(){
        //write codes here
    }
}
class Point3D extends Point2D{
    int z;
    //write codes here
}
```

```
public class Test{
    public static void main(String args[]){
        Point2D p2D = new Point2D(1, 2);
        System.out.println(p2D.Display());
        Point3D p3D = new Point3D(5, 4, 3);
        System.out.println(p3D.Display());
    }
}
```

Now:

- I. Complete the “Display” method of the Point2D class that prints all the instance variables,



Spring 2024 Question 1

```
public String Display() { 3 usages 1 override
    String s = new String( original: "x = " + x + ", y = " + y);
    return s;
}
```



Spring 2024 Question 3

QUESTION 3

[7.5 MARKS]

Write the output of the following program:

```
class Person{
    int id;
    String name;
    static int s = 10;
    {
        System.out.println("3");
    }
    public Person(){
        this.id = 1;
        this.name = "M";
        System.out.println("1");
        s++;
    }
}
```

```
public Person(int id, String name){
    this();
    this.id = id;
    this.name = name;
    System.out.println("2");
    s++;
}

public static void main(String args[]){
    Person p = new Person(1, "N");
    Person p1 = new Person();
    System.out.println(p1.s);
    p.s = 11;
    System.out.println(Person.s);
}
}
```



Spring 2024 Question 3

QUESTION 3

[7.5 MARKS]

Write the output of the following program:

<pre>class Person{ int id; String name; static int s = 10; { System.out.println("3"); } public Person(){ this.id = 1; this.name = "M"; System.out.println("1"); s++; } }</pre>	<pre>public Person(int id, String name){ this(); this.id = id; this.name = name; System.out.println("2"); s++; } public static void main(String args[]){ Person p = new Person(1, "N"); Person p1 = new Person(); System.out.println(p1.s); p.s = 11; System.out.println(Person.s); } }</pre>
--	--

Output

3
1
2
3
1
13
11



Fall 2023 Question 1

- Q1:** (a) Logarithms are mathematical functions of the form $\log_b x$ which consists of two parts: base (int b) and argument (double x). The logarithmic value is calculated using the formula: [3+3]

$$\log_b x = \frac{\log_e x}{\log_e b}$$

where $\log_e x$ can be evaluated by Java code as `Math.log(x)`.

You have to write a class **Logarithm** and include the **member** variables b and x, different types of **constructors** and a function **myfunc()** to evaluate $\log_b x$ such that the **Main** generates the output provided in the following table:

	Output
<pre>public class Main { public static void main(String[] args) { Logarithm log1 = new Logarithm(2, 9); Logarithm log2 = new Logarithm(log1); Logarithm log3 = new Logarithm(); System.out.println(log1.b + " "+log1.x+" "+log1.myfunc()); System.out.println(log2.b + " "+log2.x+" "+log2.myfunc()); System.out.println(log3.b + " "+log3.x+" "+log3.myfunc()); } }</pre>	2 9.0 3.0 2 9.0 3.0 0 0.0 0.0

```
public class Logarithm { 7 usages
    int b; 9 usages
    double x; 9 usages

    public Logarithm(int b, double x) { 1 usage
        this.b = b;
        this.x = x;
    }

    public Logarithm(Logarithm log) { 1 usage
        this.b = log.b;
        this.x = log.x;
    }

    public Logarithm() { 1 usage
        this.b = 0;
        this.x = 0.0;
    }

    public double myfunc() { 3 usages
        if (b == 0 || x == 0) {
            return 0.0;
        }
        return Math.log(x) / Math.log(b);
    }
}
```



Fall 2023 Question 2

Q2: (a) Consider the following codes

[3+3]

```
public class BankAccount {  
    public String name;  
    private String account_id;  
    private double balance;  
    BankAccount(String name,double balance,  
                String gender){  
        this.name=name;  
        this.balance=balance;  
        this.account_id=gender+"-"+name;  
    }  
    //Add necessary codes here  
}
```

```
class Test{  
    public static void main(String[] args) {  
        BankAccount b=new  
            BankAccount("Mr.Rahman",1000, "M");  
        System.out.println("Account id:"+b.account_id);  
        System.out.println("balance before:"+b.balance);  
        b.balance = b.balance - 2000;  
    }  
}
```

Rewrite the codes after correcting the errors by adding necessary getter/setter methods to access private variables. **Do not change access modifiers** of member variables. While updating the balance, a condition that **balance cannot be less than 0** should be considered.



Fall 2023 Question 2

```
public class BankAccount { 2 usages
    public String name;
    private String account_id; 2 usages
    private double balance; 3 usages

    public BankAccount(String name, double balance, String gender) { 1 usage
        this.name = name;
        this.balance = balance;
        this.account_id = gender + "-" + name;
    }

    public String getAccountId() { 1 usage
        return account_id;
    }

    public double getBalance() { 2 usages
        return balance;
    }

    public void setBalance(double balance) { 1 usage
        if (balance >= 0) {
            this.balance = balance;
        }
    }
}
```



Static and Final



Static Variables

- When a member (both methods and variables) is declared static, it can be accessed before any objects of its class are created, and without reference to any object.
- **Static variable**
 - Instance variables declared as static are like global variables.
 - When objects of its class are declared, no copy of a static variable is made.



Static Methods & Blocks

▣ **Static method**

- ▣ They can only call other static methods.
- ▣ They must only access static data.
- ▣ They cannot refer to **this** or **super** in anyway.

▣ **Static block**

- ▣ Initialize static variables.
- ▣ Get executed exactly once, when the class is first loaded.



Example

```
3 public class StaticTest {
4     static int a = 3, b;
5     int c;
6
7     static void f1(int x) {
8         System.out.println("x = " + x);
9         System.out.println("a = " + a);
10        System.out.println("b = " + b);
11        // System.out.println("c = " + c); // Error
12    }
13    int f2() {
14        return a*b;
15    }
16    static {
17        b = a*4;
18        // c = b; // Error
19    }
20    public static void main(String[] args) {
21        f1(42); // StaticTest.f1(84);
22        System.out.println("b = " + b);
23        //System.out.println("Area = " + f2()); // Error
24    }
25 }
```



Final

- ❑ Declare a final variable, prevents its contents from being modified.
- ❑ Final variable must initialize when it is declared.
- ❑ It is common coding convention to choose all uppercase identifiers for final variables
 - ❑ `final int FILE_NEW = 1;`
 - ❑ `final int FILE_OPEN = 2;`
 - ❑ `final int FILE_SAVE = 3;`
 - ❑ `final int FILE_SAVEAS = 4;`
 - ❑ `final int FILE_QUIT = 5;`



SCOPE & LIFETIME OF VARIABLES



SCOPE OF VARIABLE

- What is scope?
 - A scope determines what variable are visible to other parts of your program. Or where the variable is accessible?
 - It also determines the lifetime of those variable.
- A block defines a *scope*.
 - the statements between opening and closing curly braces.
- **As a general rule**, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.
- Within a block, variables can be declared at any point, but are valid only after they are declared.
 - Thus, if you define a variable at the start of a method, it is available to all of the code within that method.
 - Conversely, if you declare a variable at the end of a block, it is effectively useless, because no code will have access to it.



SCOPE OF VARIABLE - EXAMPLE

```
public void calculateInterest(double balance)  
{  
    if(balance > 10000)  
    {  
        float interest = 0.05f;  
    }  
    else  
    {  
        interest = 0.02f;  
    }  
}
```



SCOPE OF VARIABLE - EXAMPLE

```
public void calculateInterest(double balance)
```

```
{  
    if(balance > 10000)  
    {  
        float interest = 0.05f; // Scope of this variable is only inside the if  
        block  
    }  
    else  
    {  
        interest = 0.02f; // compiler error. interest is declared inside the if  
        block, hence can't access in else block  
    }  
}
```

- To make “interest” accessible to both if and else block it has to be declared outside of the block.



SCOPE OF VARIABLE - EXAMPLE

```
public void calculateInterest(double balance)  
{  
    float interest; // accessible to anywhere inside the method.  
    if(balance > 10000)  
    {  
        interest = 0.05f; // Ok  
    }  
    else  
    {  
        interest = 0.02f; // OK  
    }  
}
```



SCOPE OF VARIABLE

- Many other computer languages define two general categories of scopes:
 - global and local.
- However, these traditional scopes do not fit well with Java's strict, object-oriented model.
- In Java, the two major scopes are
 - those defined by a class and
 - those defined by a method.



WHEN CAN 2 VARIABLES HAVE SAME NAME

- ❑ Instance variable and Local variable.
- ❑ Local variable in 2 different methods.
- ❑ 2 Local variables in the same methods but only after the death of one Local variable.

OK	Wrong
<pre>public void calculateInterest(double balance) { if(balance > 10000){ float interest = 0.05f; // OK } else { float interest = 0.02f; // OK } }</pre>	<pre>public void calculateInterest(double balance) { float interest; if(balance > 10000){ float interest = 0.05f; // Compiler error } else { interest = 0.02f; // OK } }</pre>

ARRAYS



ARRAY AGAIN

- What is the “new” keyword during array creation.
- Is Array variable reference type?

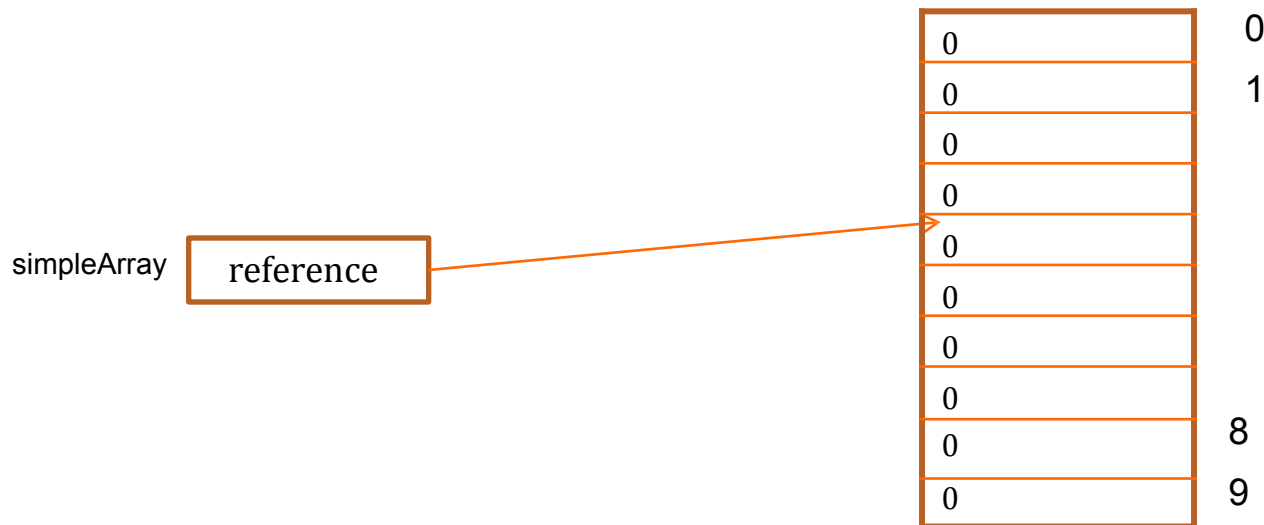
```
int[] sampleArray = new int[10];
```



ARRAY AGAIN

- What is the “new” keyword during array creation.
- Is Array variable reference type?

```
int[] sampleArray = new int[10];
```



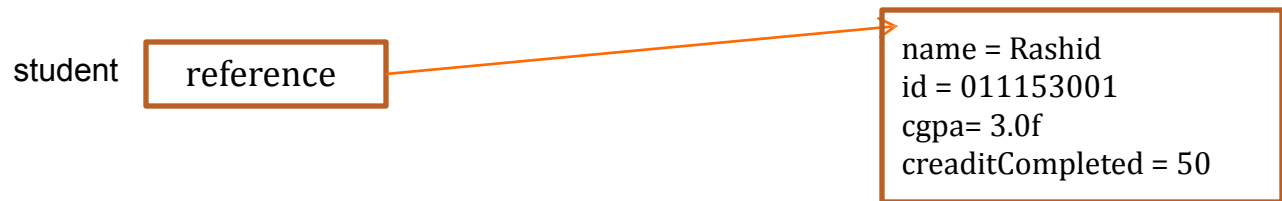
ARRAY AGAIN

- When an array is created, each element will be initialized to its default value.
- What is the initial value for each of the element of the arrays below.
 - `int[] sampleArray = new int[10];`
 - `Student[] students = new Student[10];`



REFERENCE TYPE WITH NOT NULL

```
Student student = new Student("Rashid", "011153001", 3.0f, 50);
```



- What value will you get when you access the following attributes of student reference variable/object.
 - student.name
 - student.id
 - student.cgpa
 - student.creditCompleted



REFERENCE TYPE WITH NULL

```
Student student = null;
```

student

null

- What value will you get when you access the following attributes of student reference variable/object.
 - student.name
 - student.id
 - student.cgpa
 - student.creditCompleted



REFERENCE TYPE WITH NULL

- We cant access any member via the reference variable when no object is created.
- Accessing the member will throw `NullPointerException`.

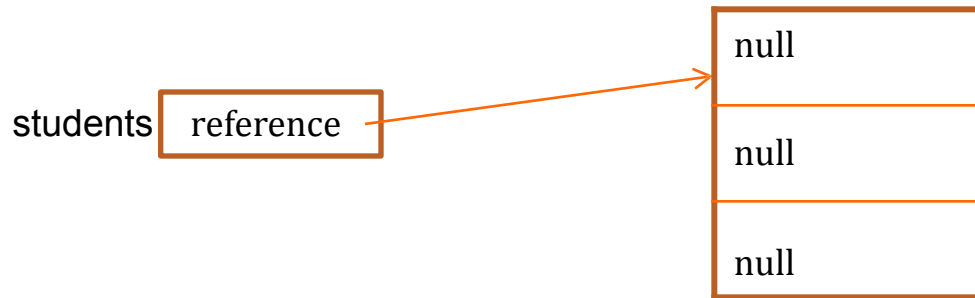


REFERENCE TYPE ARRAY

□ Example

```
Student[] students = new Student[3];
```

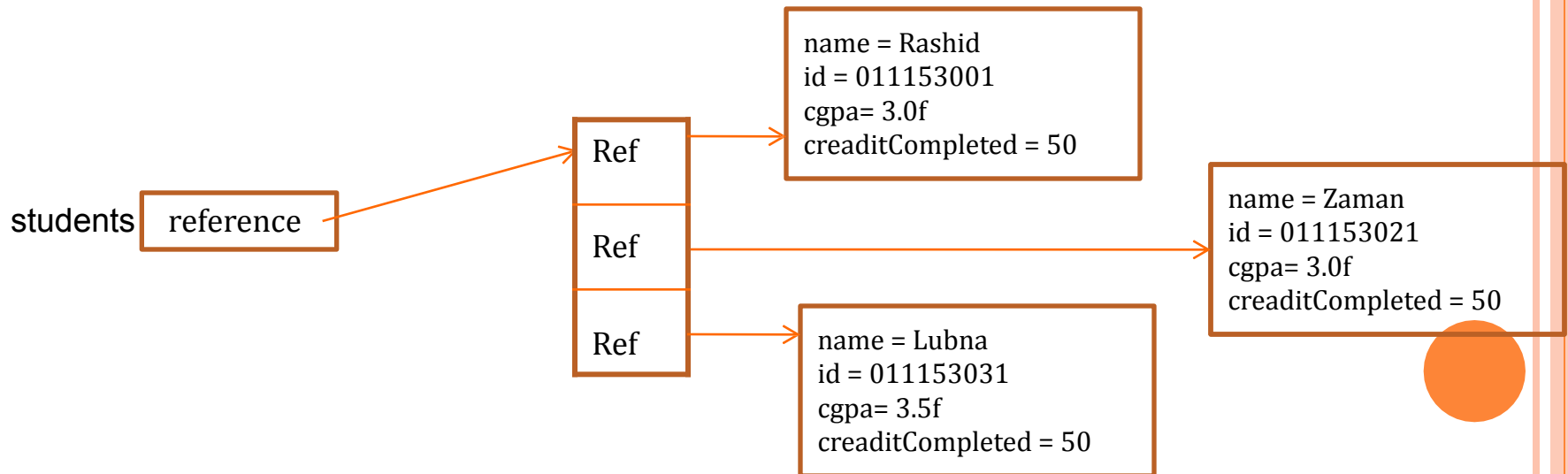
`System.out.println(students[0].cgpa);` // What would be the output of this line.



REFERENCE TYPE ARRAY

- Need to initialize the element before accessing.
- Example

```
Student[] students = new Student[3];  
students[0] = new Student("Rashid", "011153001", 3.0f, 50);  
students[1] = new Student("Zaman", "011153021", 3.0f, 50);  
students[2] = new Student("Lubna", "011153031", 3.5f, 50);  
System.out.println(students[0].cgpa); // What would be the output of this line.
```



PARAMETER PASSING



PARAMETER PASSING

- 2 different ways
 - Pass By Value
 - Pass By Reference
- In **Java** all parameters are **passed by value**



PASS BY REFERENCE – NOT APPLICABLE FOR JAVA

- Send the location of the parameter.
- Original value change
- C, C++, php

```
main() {  
    int i = 10, j = 20;  
    cout << i << " " << j << endl;  
    swapThemByRef(i, j);  
    cout << i << " " << j << endl; // displays 20 10 ...  
}  
void swapThemByRef(int& num1, int& num2) { int  
    temp = num1;  
    num1 = num2;  
    num2 = temp;  
}
```



PASS BY REFERENCE – NOT APPLICABLE FOR JAVA

- ❑ Send the location of the parameter.
- ❑ Original value change
- ❑ C, C++, php

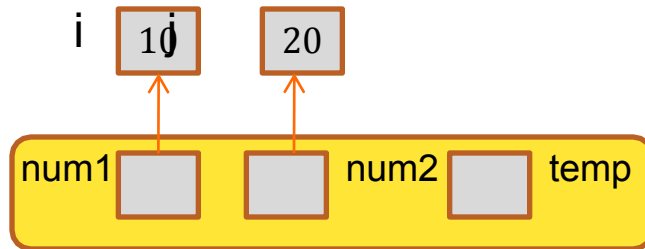
```
main() {  
    int i = 10, j = 20;  
    cout << i << " " << j << endl;  
    swapThemByRef(i, j);  
    cout << i << " " << j << endl; // displays 20 10 ...  
}  
void swapThemByRef(int& num1, int& num2) { int  
    temp = num1;  
    num1 = num2;  
    num2 = temp;  
}
```

Output:
10 20
20 10

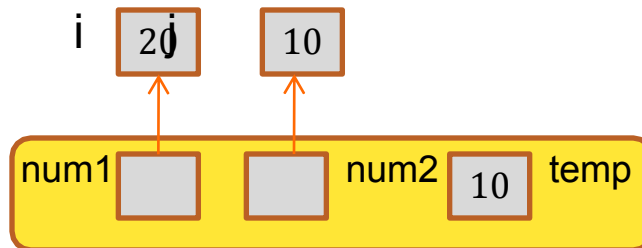


PASS BY REFERENCE

- At the beginning of function call



- After the function execution.



PASS BY VALUE

- Send a copy of the original parameter.
- Original value does not change
- Example:

```
main() {  
    int i = 10, j = 20;  
    cout << i << " " << j << endl;  
    swapThemByVal(i, j);  
    cout << i << " " << j << endl;  
}  
void swapThemByVal(int num1, int num2) { int  
    temp = num1;  
    num1 = num2;  
    num2 = temp;  
}
```



PASS BY VALUE

- Send a copy of the original parameter.
- Original value does not change
- Example:

```
main() {  
    int i = 10, j = 20;  
    cout << i << " " << j << endl;  
    swapThemByVal(i, j);  
    cout << i << " " << j << endl; // displays 20 10 ...  
}  
  
void swapThemByVal(int num1, int num2) { int  
    temp = num1;  
    num1 = num2;  
    num2 = temp;  
}
```

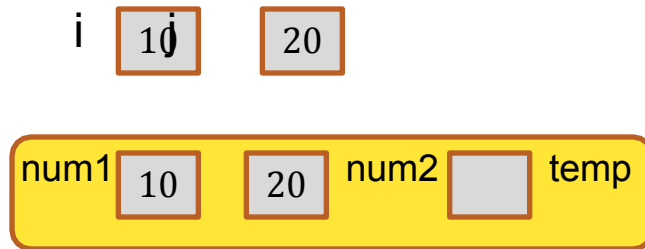
Output:

```
10 20  
10 20
```

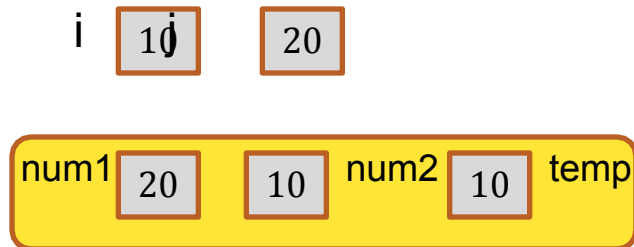


PASS BY VALUE

- At the beginning of method call



- After the method execution.



PASS BY VALUE

Java Example:

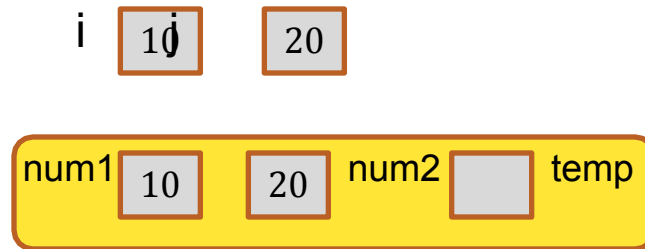
```
public class PassByValue {  
    public static void main(String[] args) {  
        int a=10, b=20;  
        System.out.printf("a-%d:b-%d\n", a, b);  
        swapThemByVal(a, b);  
        System.out.printf("a-%d:b-%d\n", a, b);  
    }  
  
    static void swapThemByVal(int num1, int num2) { int  
        temp = num1;  
        num1 = num2;  
        num2 = temp;  
    }  
}
```

Output:
a-10:b-20
a-10:b-20

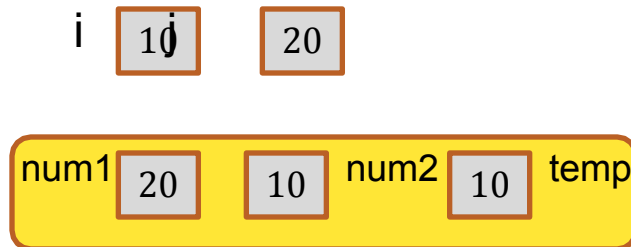


PASS BY VALUE

- At the beginning of method call



- After the method execution.



PASS BY VALUE – WITH OBJECT

- When an object is passed to a method, the situation changes dramatically,
- Java will pass by value but the effect will be like pass-by-reference.
- Why?
 - When we create a variable of a class type, it will store the reference to an object.
 - Thus, when the value of the variable will be passed to a method, it will pass the reference of the same object.
 - This effectively means that objects act as if they are passed to methods by use of pass-by-reference.
 - Changes to the object inside the method *do affect the object used as an argument*.



PASS BY VALUE – WITH OBJECT

```
public class Test{  
    String testName;  
    float score;  
  
    Test(String n, float s){  
        testName = n; score  
        = s;  
    }  
  
    void display(){  
        System.out.printf("TestName: %s ; Score: %.2f\n", testName,  
        score);  
    }  
}
```



PASS BY VALUE – WITH OBJECT

```
public class PassByValue {  
    public static void main(String[] args) {  
        Test t = new Test("CT1", 10);  
        t.display();  
        updateScore(t, 15.0f);  
        System.out.println("After Update:");  
        t.display();  
    }  
  
    static void updateScore(Test test, float newScore) {  
        test.score = newScore;  
    }  
}
```

Output:

TestName: CT1 ; Score: 10.00

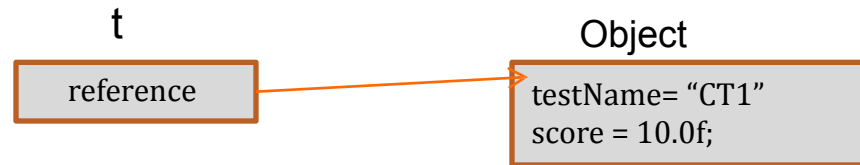
After Update:

TestName: CT1 ; Score: 15.00



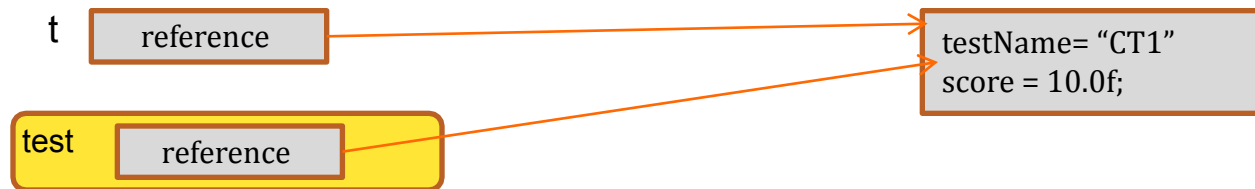
PASS BY VALUE – WITH OBJECT

- Before the method call

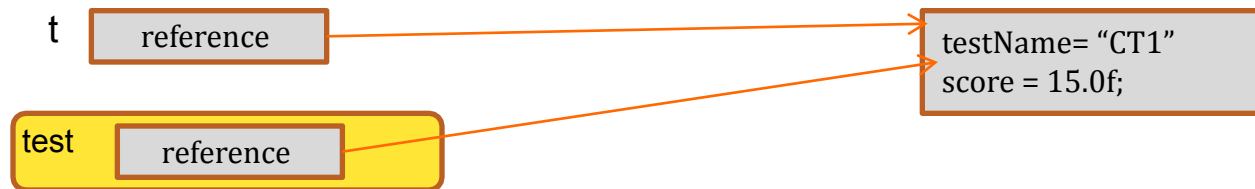


- At the beginning of method call

- Both `"t"` and `"test"` are referring to the same object.
- Updating the object using any variable will be reflected in the other one.



- Just at the end of method execution – before exiting the method



GARBAGE COLLECTION



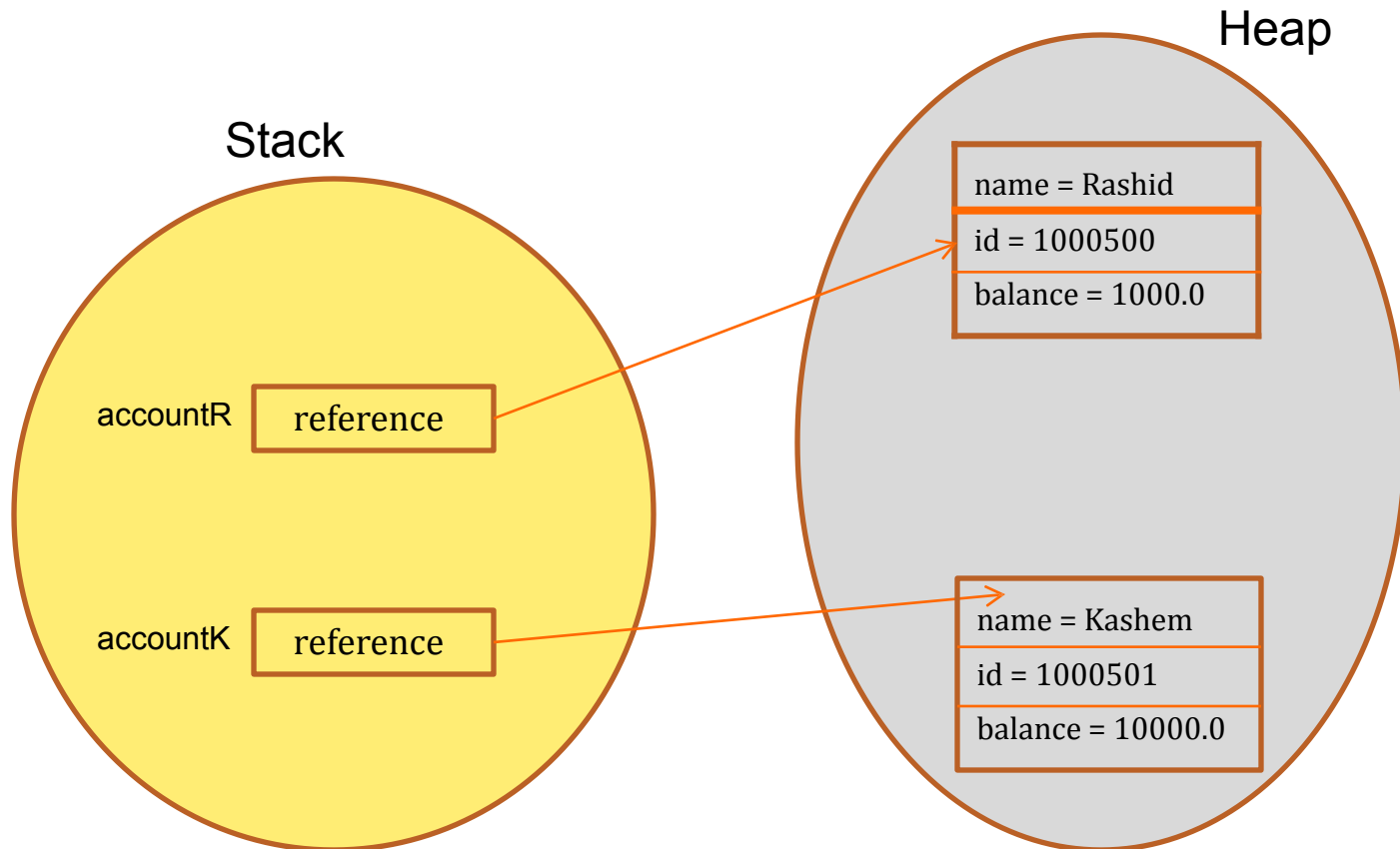
GARBAGE COLLECTION

- when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.
- Garbage collection only occurs sporadically (if at all) during the execution of your program.
- Different Java run-time implementations will take varying approaches to garbage collection,
- but for the most part, you should not have to think about it while writing your programs.



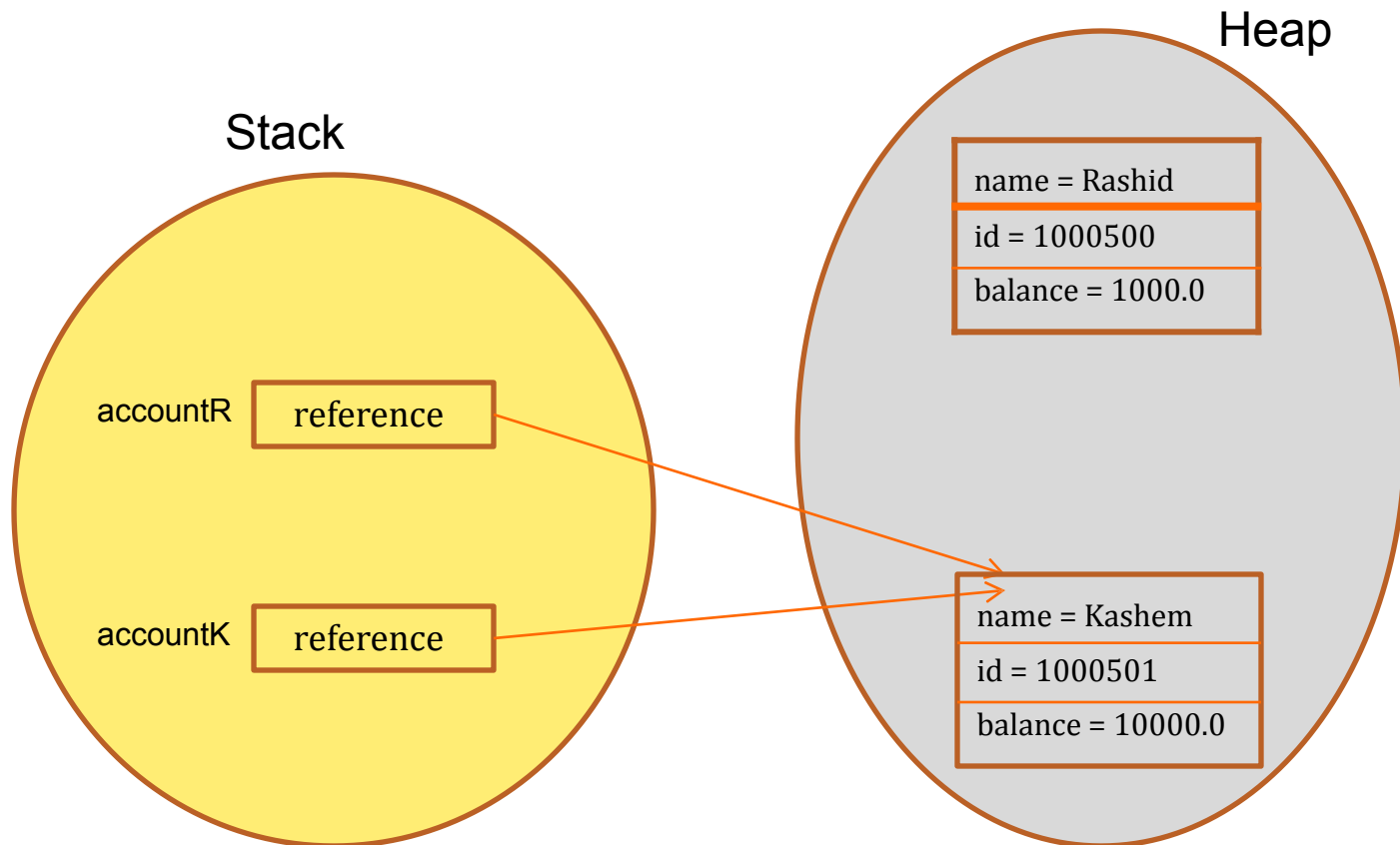
GARBAGE COLLECTION – SCENARIO#1

```
BankAccount accountR = new BankAccount("Rashid", "1000500", 1000.0);  
BankAccount accountK = new BankAccount("Kashem", "1000501", 10000.0);
```



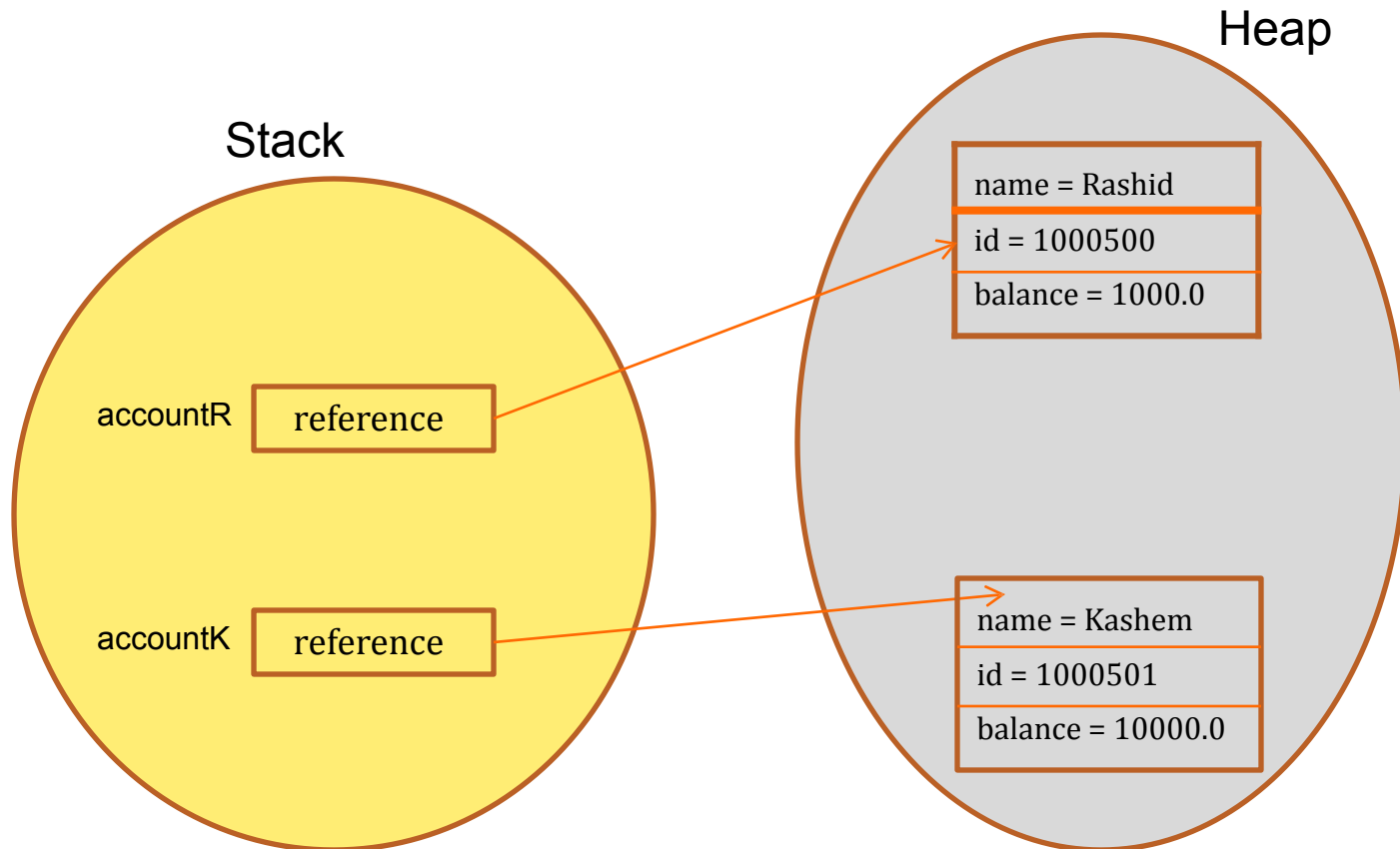
GARBAGE COLLECTION – SCENARIO#1

```
BankAccount accountR = new BankAccount("Rashid", "1000500", 1000.0);  
BankAccount accountK = new BankAccount("Kashem", "1000501", 10000.0);  
accountR = accountK;  
// Rashid's object can no longer be accessed and is eligible for garbage collection
```



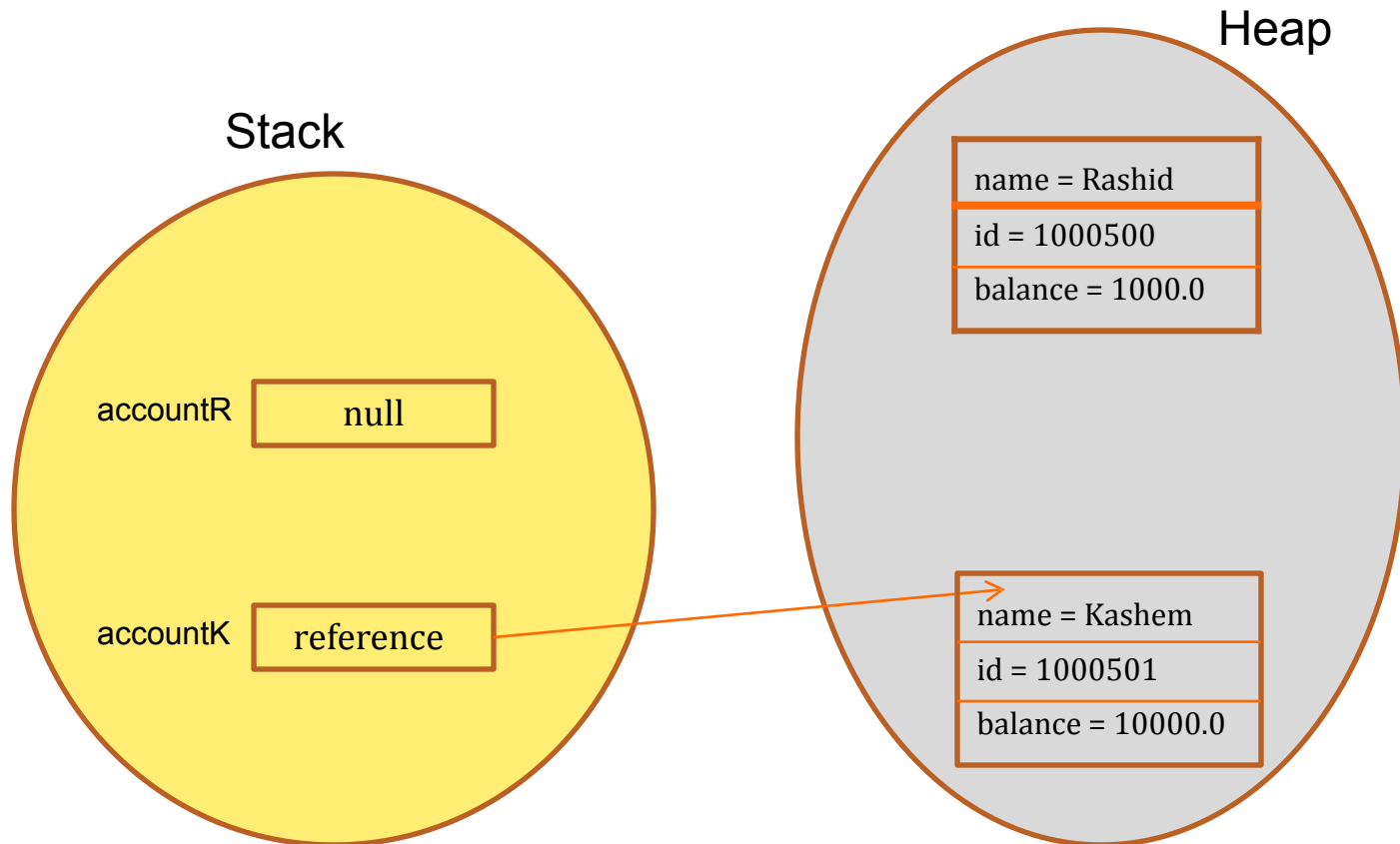
GARBAGE COLLECTION – SCENARIO#2

```
BankAccount accountR = new BankAccount("Rashid", "1000500", 1000.0);  
BankAccount accountK = new BankAccount("Kashem", "1000501", 10000.0);
```



GARBAGE COLLECTION – SCENARIO#2

```
BankAccount accountR = new BankAccount("Rashid", "1000500", 1000.0);  
BankAccount accountK = new BankAccount("Kashem", "1000501", 10000.0);  
accountR = null;  
// Rashid's object can no longer be accessed and is eligible for garbage collection
```



GARBAGE COLLECTION – SCENARIO#3

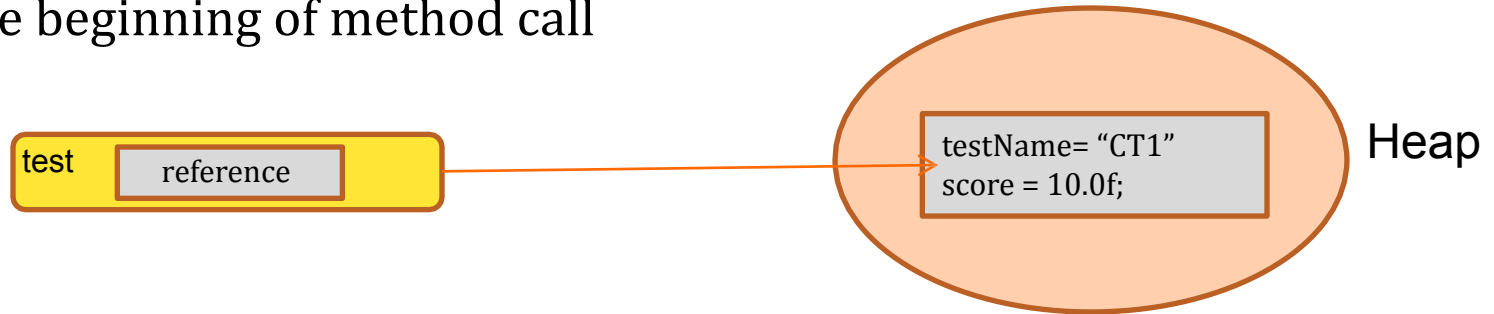
```
public class TestMain{  
    public static void main(String[] args) {  
        updateScore(new Test("CT1", 10), 15.0f);  
    }  
  
    static void updateScore(Test test, float newScore) {  
        test.score = newScore;  
    }  
}
```

```
public class Test{  
    String testName;  
    float score;  
  
    Test(String n, float s){  
        testName = n;  
        score = s;  
    }  
}
```

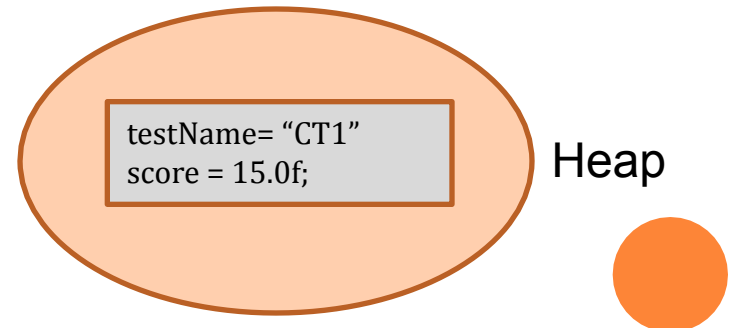


GARBAGE COLLECTION – SCENARIO#3

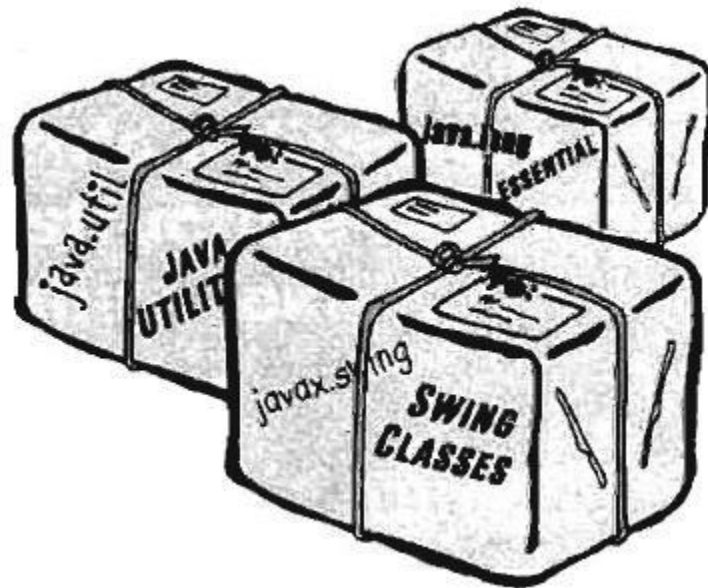
- At the beginning of method call



- After exiting the method.
 - “test” variable will no longer be available.
 - The Test object can no longer be accessed and is eligible for garbage collection

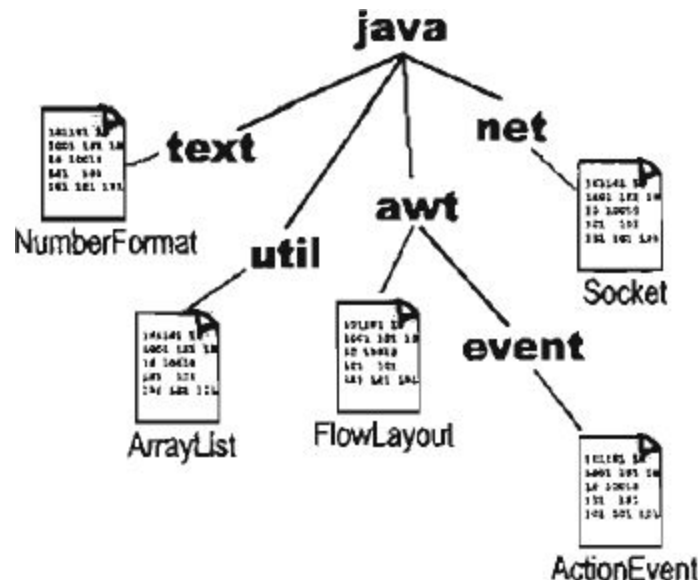


PACKAGE



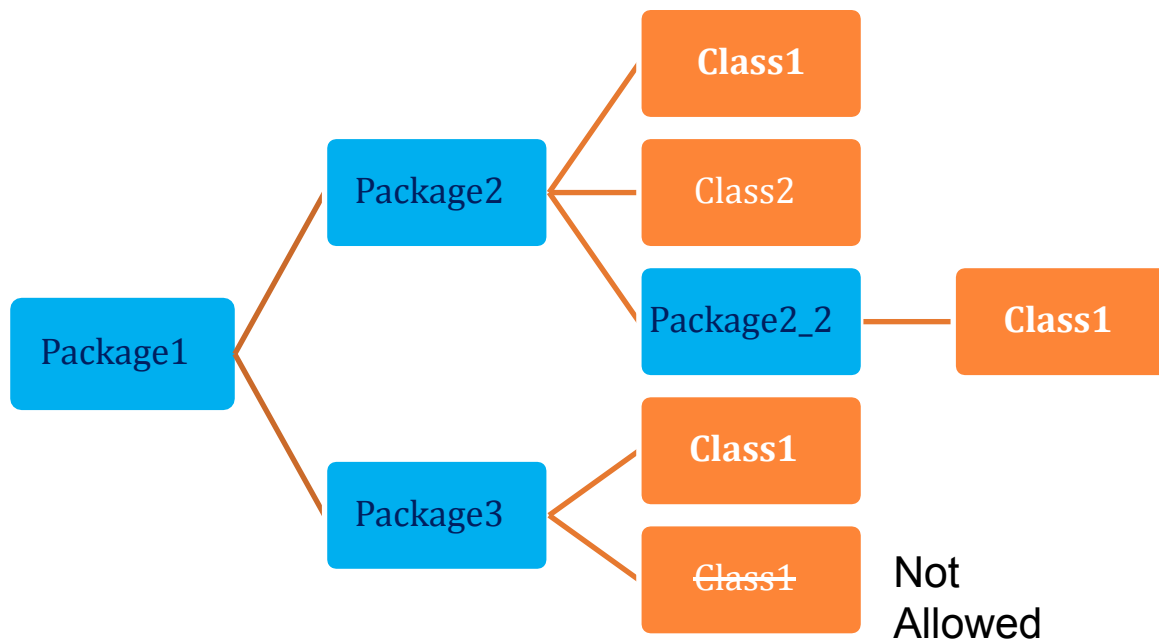
WHAT IS PACKAGE?

- ❑ Packages are used to group related classes.
- ❑ A package is a namespace that organizes a set of related classes and interfaces.
- ❑ Conceptually you can think of packages as being similar to different folders on your computer.



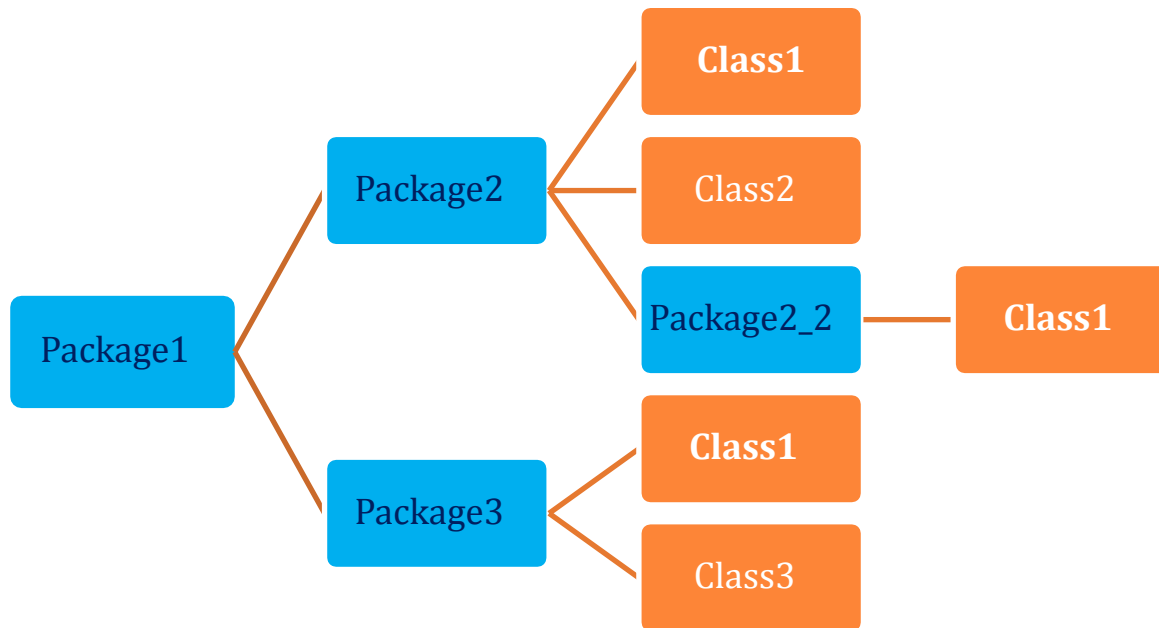
WHAT IS PACKAGE?

- ❑ Classes in same package can not have duplicate name.
- ❑ Classes in different packages can have the same name.



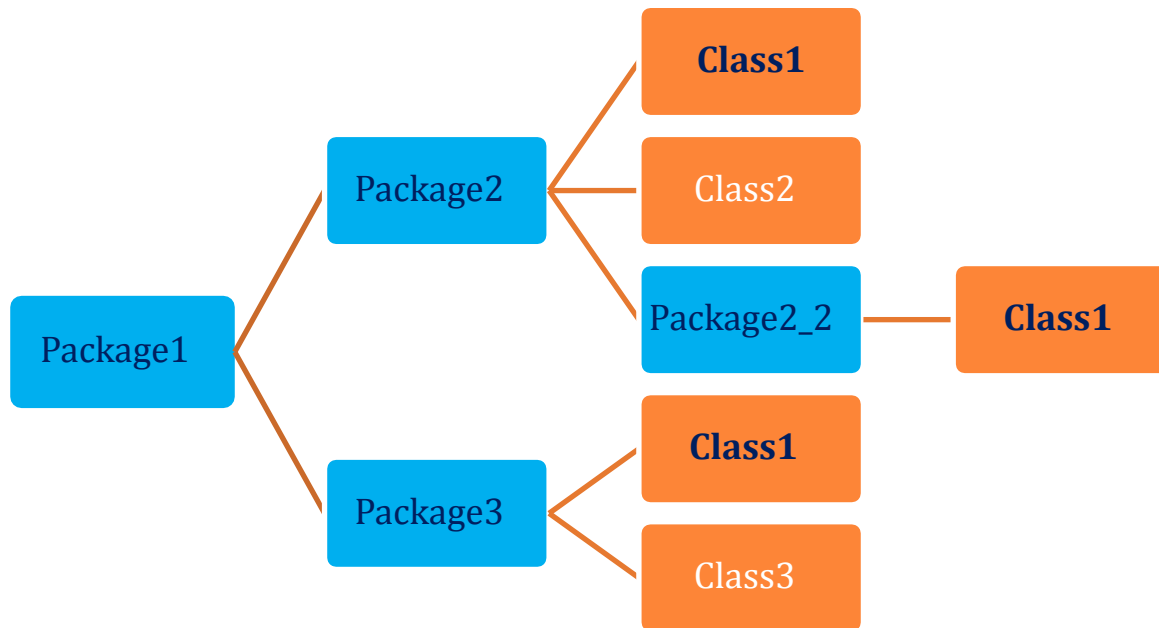
WHAT IS PACKAGE?

- ❑ Classes in same package can not have duplicate name.
- ❑ Classes in different packages can have the same name.



WHAT IS PACKAGE?

- ❑ Classes in same package can not have duplicate name.
- ❑ Classes in different packages can have the same name.



HOW TO CREATE PACKAGE?

□ To create a package is quite easy:

- simply include a **package command as the first statement** in a Java source file.

```
package pkg;
```

- Any classes declared within that file will belong to the specified package.
- Java uses **file system directories to store packages**.

□ You can create a hierarchy of packages.

- Use period/dot to separate each package name from the one above it.

```
package pkg1.pkg2.pkg3;
```



PACKAGE -EXAMPLE

□ Example:

```
package uiu.cse;  
public class Test{  
    public void display() {  
        System.out.println( "Hello for Test class." );  
    }  
}
```

1. The class must be in a file named "Test.java"
2. Place the file "Test.java" in a directory called "cse"
3. Place directory "cse" in a directory called "uiu".
4. The directory "cse" can be placed anywhere, but you need to set the classpath.



PACKAGE -EXAMPLE

- If you use IDE,
 - 1-3 will be done automatically and “cse” will be placed under the “src” folder.
 - If no package is specified the file will be placed in a default package which maps to “src” folder



BENEFITS OF USING PACKAGE

- The package is both a naming and a visibility control mechanism.
- Packages are important for three main reasons.
 - **First**, they help the overall organization of a project or library.
 - Can organize code in a logical manner
 - makes large software projects easier to manage



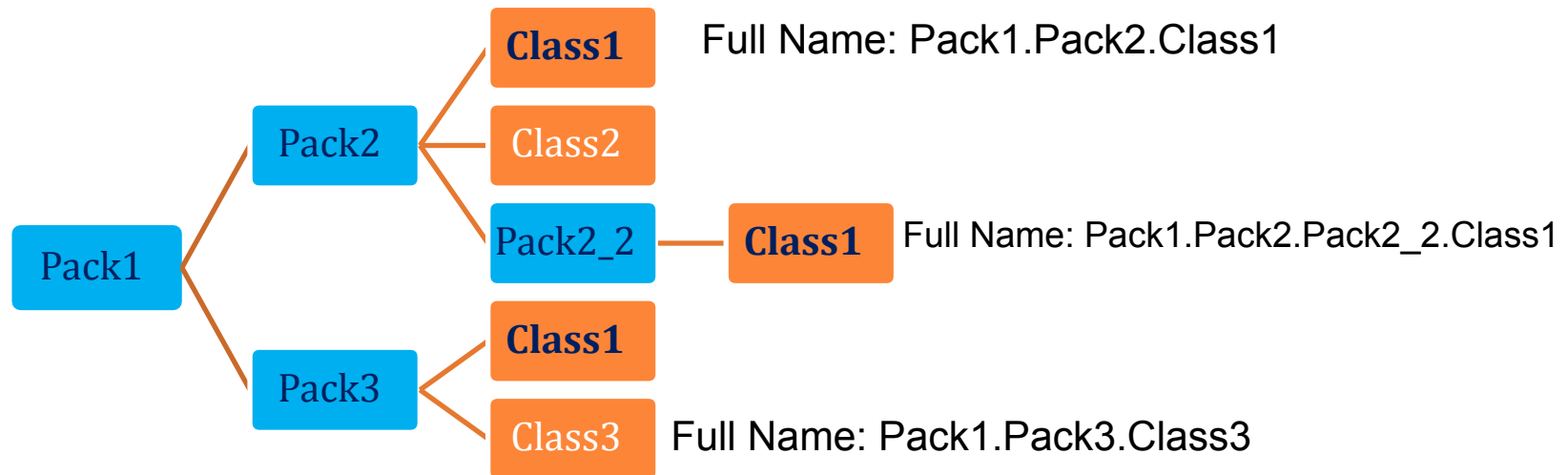
BENEFITS OF USING PACKAGE

- ▣ **Second**, packages give you a name scoping, to help prevent collisions.
 - What will happen if you and 12 other programmers in your company all decide to make a class with the same name.
- ▣ **Third**, packages provide a level of security,
 - Can define **classes** inside a package that are not accessible by code outside that package.
 - can also define class **members** that are exposed only to other members of the same package.



CLASS'S FULL NAME

- A class has a full name, which is a combination of the package name and the class name.



HOW TO ACCESS CLASS

- To use a class in same package, you can use the class name (short name).
- To use a class in a different package, you must tell Java the full name of the class.
 - You use either an Import statement at the top of your source code, and use short name or
 - you can type the full name every place you use the class in your code.
- **java.lang.*** package is always implicitly get imported by Java. [explicit import is not needed for this package]



CLASS IN SAME PACKAGE

□ Test Class

```
package uiu.cse;  
public class Test{  
    public void display() {  
        System.out.println( "Hello for Test class." );  
    }  
}
```

□ Main Class

```
package uiu.cse;  
public class TestMain {  
    public static void main(String[] args) {  
        Test test = new Test(); test.display();  
    }  
}
```



CLASS IN DIFFERENT PACKAGE – FULL NAME

□ Test Class

```
package uiu.cse;  
public class Test{  
    public void display() {  
        System.out.println( "Hello for Test class." );  
    }  
}
```

□ Main Class

```
package uiu.cse.test; // any package other than uiu.cse  
  
public class TestMain {  
    public static void main(String[] args) {  
        uiu.cse.Test test = new uiu.cse.Test(); // use Full name  
        test.display();  
    }  
}
```



CLASS IN DIFFERENT PACKAGE - IMPORT

□ Test Class

```
package uiu.cse;
public class Test{
    public void display() {
        System.out.println( "Hello for Test class." );
    }
}
```

□ Main Class

```
import uiu.cse.Test; // Need to import the class.
public class TestMain {
    public static void main(String[] args) { Test
        test = new Test(); // use Short name
        test.display();
    }
}
```

- To import all classes under a package we need to use * after the package name. Example:

```
import uiu.cse.*;
```



ACCESS CONTROL/ACCESS MODIFIER

- How a member can be accessed is determined by the *access modifier attached to its* declaration.
- 4 types
 - public
 - Protected
 - Default/Package Access – No modifier
 - Private
- Outer **class** can only be declared as **public** or **default**.



PRIVATE

- Members declared private are accessible only in the class itself
- Example:

```
class Private {  
    private String name = "Private";  
    public void print() {  
        System.out.println( name );  
    }  
}  
  
class PrivateExample {  
    public static void main( String[] args ) {  
        Private pr = new Private(); pr.print();  
        // OK  
        System.out.println( pr.name ); // Compile error  
    }  
}
```



DEFAULT/PACKAGE ACCESS

- When no access modifier is specified
- Accessible in the package that contains the class
- Not accessible outside the package that contains the class.
 - Not even child class.



DEFAULT/PACKAGE ACCESS - EXAMPLE

Class that will be accessed from other classes

```
package test;  
class Default{  
    String name = "Default";  
}
```

Class under different package

```
package test1;  
class DefaultExample {  
    public static void main( String[] args ) {  
        Default dfl= new Default();  
        System.out.println( dfl.name ); // Compile error  
    }  
}
```

Class under same package

```
package test;  
class DefaultExample1 {  
    public static void main( String[] args ) {  
        Default dfl = new Default();  
        System.out.println( dfl.name ); // OK  
    }  
}
```



DEFAULT/PACKAGE ACCESS - EXAMPLE

Child Class under different package

```
package test1;  
class DefaultChild extends Default {  
    public DefaultChild() {  
        name = "Child"; // Compile error  
    }  
}
```

Child Class under same package

```
package test;  
class DefaultChild1 extends Default {  
    public DefaultChild1() {  
        name = "Child"; // OK  
    }  
}
```



PROTECTED

- Members declared protected are directly accessible to any subclasses,
 - Even if the child is in different package
- directly accessible by code in the same package.



PROTECTED - EXAMPLE

Class that will be accessed from other classes

```
package test;  
class Protected {  
    protected String name = "Protected";  
}
```

Class Under Different Package

```
package test1;  
class ProtectedExample {  
    public static void main( String[] args ) {  
        Protected pr = new Protected();  
        System.out.println( pr.name ); // Compile error  
    }  
}
```

Class Under Same Package

```
package test;  
class ProtectedExample1 {  
    public static void main( String[] args ) {  
        Protected pr = new Protected();  
        System.out.println( pr.name ); // OK  
    }  
}
```



PROTECTED - EXAMPLE

Child Class Under Different Package

```
package test1;  
class ProtectedChild extends Protected {  
    public ProtectedChild() {  
        pr.name = "Child"; // OK  
    }  
}
```

Child Class Under Same Package

```
package test;  
class ProtectedChild1 extends Protected {  
    public ProtectedChild1() {  
        pr.name = "Child"; // OK  
    }  
}
```



PUBLIC

- ❑ Members declared public are accessible anywhere the class is accessible
- ❑ All the scenario described in other 3 types of access modifier won't give any compile error



ACCESS MODIFIER CHART

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes



RECURSION

- It is the process of a method calling itself. A recursion has two parts
 - stop condition
 - call itself
- Recursion is an expensive procedure and it is better to avoid recursion if you do not need.
- Example: Factorial, Fibonacci

```
public class MyMathLibrary { public
    static int factorial(int n){
        if (n <= 0)
            return 1; // stop condition
        return n * factorial(n-1); // call itself
    }
    public static void main( String[] args ){
        System.out.print("Factorial of 5 is: " + factorial(5));
        System.out.print("Factorial of 6 is: " + factorial(6));
    }
}
```



VARIABLE ARGUMENT



VARIABLE ARGUMENT – OLD APPROACH

// Use an array to pass a variable number of arguments to a method.

// This is the old-style approach to variable-length arguments.

```
class PassArray {  
    static void vaTest(int v[]) {  
        System.out.print("Number of args: " + v.length + " Contents: ");  
        for(int x : v)  
            System.out.print(x + " ");  
        System.out.println();  
    }  
    public static void main(String args[]) {  
        // Notice how an array must be created to hold the arguments.  
        int n1[] = { 10 };  
        int n2[] = { 1, 2, 3 };  
        int n3[] = { };  
        vaTest(n1); // 1 arg  
        vaTest(n2); // 3 args  
        vaTest(n3); // no args  
    }  
}
```

Output:

```
Number of args: 1 Contents: 10  
Number of args: 3 Contents: 1 2  
3 Number of args: 0 Contents:
```



VARIABLE ARGUMENT – NEW APPROACH

- ❑ Introduced in Java 5.
- ❑ Can pass variable number of argument.
- ❑ This feature is called ***varargs*** - short for *variable-length arguments*.
- ❑ *A method that takes a variable number of arguments is called a variable-arity method, or simply a varargs method.*
- ❑ A variable-length argument is specified by three periods (...).
- ❑ Example : `static void vaTest(int ... v) {`
- ❑ The argument `v` act as an array



VARARG - EXAMPLE

```
// Demonstrate variable-length arguments.
class VarArgs {
    // vaTest() now uses a vararg.
    static void vaTest(int ... v) {
        System.out.print("Number of args: " + v.length + " Contents: ");
        for(int x : v)
            System.out.print(x + " ");
        System.out.println();
    }
    public static void main(String args[]) {
        // Notice how vaTest() can be called with a
        // variable number of arguments.
        vaTest(10); // 1 arg
        vaTest(1, 2, 3); // 3 args
        vaTest(); // no args
    }
}
```

Output: Same as before



VARIABLE ARGUMENT

- A method can have “normal” parameters along with a variable-length parameter.
- However, the variable-length parameter must be the last parameter declared by the method.
- For example,

- Following method declaration is perfectly acceptable:

```
int doIt(int a, int b, double c, int ... vals) {
```

- Inacceptable method

```
int doIt(int a, int b, double c, int ... vals, boolean stopFlag) { // Error!
```

- Can't have more than one vararg parameter

```
int doIt(int a, int b, double c, int ... vals, double ... morevals) { // Error!
```



REFERENCE

- ▣ Java:Complete Reference Chapter 6,7,9

